

DishonorPredict: Production MLOps Pipeline

Automated Serverless Cheque Bounce Classification Engine

Core Framework: GitHub Actions CI/CD, Random Forest Topology, Target Encoding, & ONNX Runtime Serialization

System Architecture & Data Isolation

DECOUPLING ANALYTICAL WORKSPACES FROM PRODUCTION RUNTIMES

- **Strict Separation of Concerns:** Isolates the model training data environment from live active evaluation lines to eliminate any potential data leakage or operational collisions.
- **Analytical Offline Layer (Historical Data):** Connects securely via service accounts to Spreadsheet ID 1NIVid94 . . . (Tab: RAW Data) to parse historical trends, compute base probabilities, and fit model weights.
- **Production Execution Layer (Operational Ingress):** Targets Spreadsheet ID 18nbA_0B . . . (Tab: dishonorPredict) to pull unlabelled production transactions and write back real-time prediction scores.

SPEAKER NOTES

"Our framework enforces strict data isolation boundaries to prevent pipeline contamination. The training phase reads once a month from an offline historical archive dataset. Conversely, our deployment engine interacts exclusively with a dedicated production tracking tab to perform in-place mathematical score updates every single morning."

Feature Engineering & Ingestion Pipeline

VECTORIZING HETEROGENEOUS STRINGS INTO DENSE INPUT MATRICES

- **Data Cleansing & Type Coercion:** Coerces currency strings and processing delays into clean continuous values, applying zero-value imputation handles for missing information.
- **Temporal Decomposition:** Deconstructs chronological elements from raw strings, explicitly capturing cyclical seasonal trends by mapping transaction months.
- **Supervised Target Encoding:** Avoids high-dimensional one-hot explosion by mapping the high-cardinality nominal variable Agent Name directly into historical conditional risk probabilities.
- **Out-of-Vocabulary (OOV) Fallbacks:** Unseen classes identified in the real-time operational stream are automatically mapped to the running global prior expectation value to ensure stable system execution.

SPEAKER NOTES

"To feed our model, the ingestion pipeline transforms raw entries into structured input matrices. We clean continuous features like amount and delay, extract seasonal identifiers from transaction dates, and utilize target encoding for high-cardinality nominal features like agent names. If an unseen agent appears during live operations, our out-of-vocabulary handler falls back to the global historical risk average."

Algorithmic Blueprint & Training Engine

MITIGATING SEVERE TARGET CLASS IMBALANCE WITH ENSEMBLES

- **Core Model Topology:** Utilizes a `RandomForestClassifier` architecture built on an ensemble of 100 bootstrap decision tree base-learners.
- **Deterministic Optimization:** Hardcodes reproducible seed values (`random_state=42`) to guarantee perfectly consistent node-splitting and sub-spacing metrics during training.
- **Addressing Highly Skewed Data:** Bounced transactions represent small minority anomalies relative to routine clearances, creating an unbalanced training distribution.
- **Cost-Sensitive Balanced Penalization:** Applies optimization weights inversely proportional to target class frequencies (`class_weight='balanced'`), forcing the model to penalize minority misclassifications aggressively.

SPEAKER NOTES

"For our classification core, we implement a Random Forest consisting of 100 estimators. Because cheque dishonor events are rare anomalies, our target distribution is highly skewed. We counteract this imbalance during training by using cost-sensitive learning—adjusting our class weights to penalize majority-class bias and capture high-risk flags effectively."

Model Serialization & Deployment Optimization

COMPILING LIGHTWEIGHT CROSS-PLATFORM GRAPH BINARIES

- **Decoupled Production Footprint:** Avoids loading the full, resource-heavy Scikit-Learn framework in production by tracing and converting the pipeline layout via `skl2onnx`.
- **Universal Model Graph Asset:** Serializes decision parameters into a highly optimized, cross-platform binary format: `rf_dishonor_model.onnx` (Opset 12 compatibility).
- **Externalized Feature Parameters:** Isolates target encoding weights, coefficient mappings, and population averages within a lightweight JSON metadata schema: `model_metadata.json`.
- **Low-Latency Runtime Execution:** Live evaluation workers deploy an optimized `onnxruntime` session to score incoming records instantly with near-zero runtime overhead.

SPEAKER NOTES

"To achieve a true serverless footprint, we don't deploy Scikit-Learn in our production runtime. Instead, we serialize our fitted parameters into a lightweight ONNX binary graph. When evaluating live records, our pipeline initializes an optimized ONNX Runtime session, allowing us to drop compute dependencies and generate predictions in milliseconds."

Classification Thresholds & Matrix Mutations

OPTIMIZING SENSITIVITY BOUNDARIES FOR OPERATIONAL FLAGS

- **Custom Decision Boundary:** Discards standard 50% default decision thresholds in favor of an aggressive 10% operational risk trigger ($P = 0.10$) to heavily prioritize sensitivity over specificity.
- **Batch Score Vectorization:** Isolates rows missing classifications, computes prediction probabilities in an array operation, and maps outputs directly across two target headers:



FLAG: High Risk (Review Transaction)

Triggered automatically when the posterior probability distribution for an event scales to 10% or higher ($P \geq 0.10$).



PASS: Low Risk

Assigned to safe operational rows whose computed probability falls safely below the 10% threshold boundaries ($P < 0.10$).

SPEAKER NOTES

"When mapping risk probabilities to our classification matrix, we reject standard 50% cut-offs. Because default costs are high, we enforce an aggressive 10% operational threshold to maximize sensitivity. Any record with a calculated probability of 10% or higher is tagged with a high-visibility alert directly within the worksheet."

MLOps Orchestration & Pipeline Automation

HANDS-OFF CI/CD INTEGRATION VIA GITHUB ACTIONS WORKFLOWS

- **Daily Live Scoring Automation (`predict.py`):** Triggered via a POSIX cron schedule every single day at 06:00 UTC (11:30 AM IST) to evaluate new rows on ephemeral ubuntu-latest runners.
- **Monthly Retraining Optimization Loop (`train.py`):** Automated cron event executing on the 1st day of every month, rewriting refreshed graph weights back to the main branch.
- **Manual Ad-Hoc Dispatches:** Integrates native `workflow_dispatch` hooks to allow system operators to manually trigger training or prediction sequences instantly via the GitHub UI.
- **Encrypted API Credentials Vault:** Protects Google Cloud sheet connections by passing identity credentials strictly within an encrypted repository secret (`GCP_SERVICE_ACCOUNT_JSON`).

SPEAKER NOTES

"Our pipeline automation uses declarative GitHub Actions workflows. A daily cron trigger runs live predictions, while a monthly cron workflow automatically retrains our model weights using the latest data additions. Everything is securely isolated, using a hidden Google Cloud Service Account credential token stored inside our repository vault. Thank you."